



합성곱 신경망 I – part.2

DL 8기 4팀 팽소원, 김혜원, 함예린

목차

#5.3 전이학습 (1)

#5.3 전이학습 (2)

#5.4 설명 가능한 CNN

#5.5 그래프 합성곱 네트워크



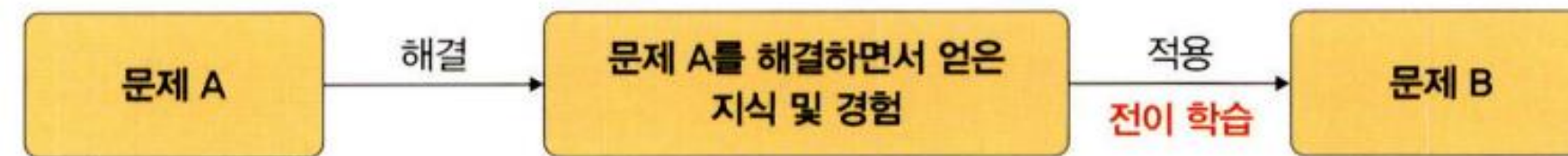
5.3 전이학습 (1)



#5.3 전이 학습

전이 학습: 이미지넷처럼 아주 큰 데이터셋을 써서 훈련된 모델의 가중치를 가져와 우리가 해결하려는 과제에 맞게 보정해서 사용하는 것

▼ 그림 5-29 전이 학습



- 합성곱 신경망 기반의 딥러닝 모델 : 많은 양의 데이터 필요 -> 큰 데이터셋 확보 어려움 -> 전이 학습 등장 !
- 사전 훈련된 모델(네트워크) : 아주 큰 데이터셋을 사용하여 훈련된 모델
- 전이 학습을 위한 방법: 1) 특성 추출 기법 2) 미세 조정 기법

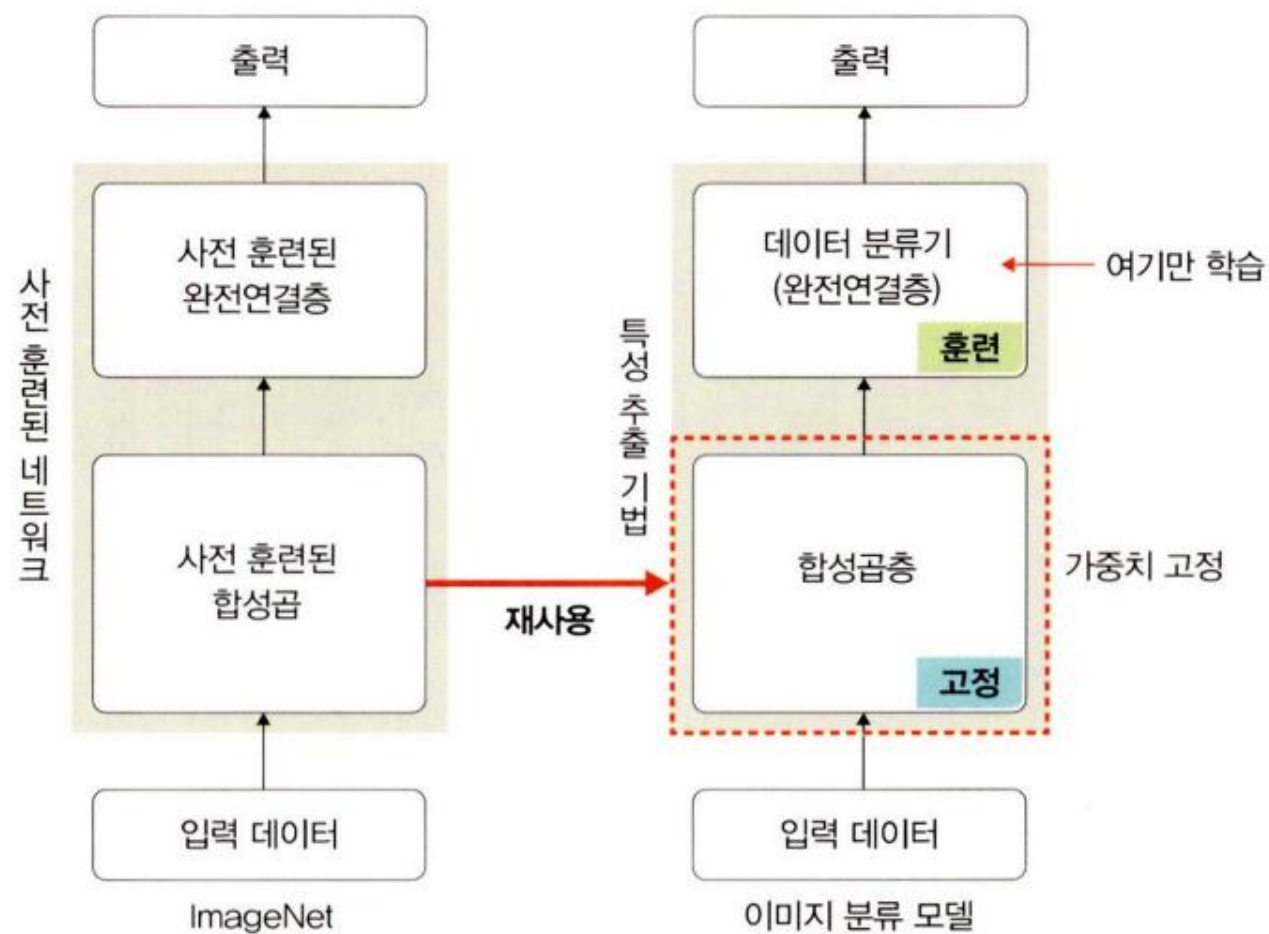


ImageNet : 2009년에 만들어진 라벨이 지정된 이미지의 대규모 데이터셋 (모델X)
20000개 이상의 카테고리에 속하는 1400만 개 이상의 이미지가 포함되어 있음

#5.3.1 특성 추출 기법

특성 추출: 이미지넷 데이터셋으로 사전 훈련된 모델을 가져온 후 마지막에 완전연결층 부분만 새로 만듦

▼ 그림 5-30 특성 추출 기법



- 이미지 분류를 위한 두 부분

- ① 합성곱층 : 합성곱층과 풀링층으로 구성
- ② 완전 연결층 : 추출된 특성을 입력받아 최종적으로 이미지에 대한 클래스를 분류하는 부분

● ImageNet으로 사전훈련된 모델:
Xception, InceptionV3, ResNet50, VGG16, VGG19, MobileNet

#5.3 전이 학습

#라이브러리 설치

> pip install opencv-python

Open CV는 Open Source Computer Vision Library의 약어

#라이브러리 호출

```
# 라이브러리 호출
import os
import time
import copy
import glob
import cv2
import shutil
import numpy as np

import torch
import torchvision # 컴퓨터 비전(computer vision) 용도의 패키지
import torchvision.transforms as transforms # 데이터 전처리를 위해 사용되는 패키지
import torchvision.models as models # 다양한 파이토치 네트워크를 사용할 수 있도록 도와주는 패키지
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader

import matplotlib.pyplot as plt
```


#5.3 전이 학습

#이미지 데이터 전처리 방법 정의

```
# 이미지 데이터 전처리 방법 정의
data_path = r'C:\Euron\Week4\catanddog\train'

transform = transforms.Compose(
    [
        transforms.Resize([256, 256]),
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor()
    ])
train_dataset = torchvision.datasets.ImageFolder(
    data_path,
    transform=transform
)
train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=32,
    num_workers=8,
    shuffle=True
)

print(len(train_dataset))
```

● torchvision.transform : 이미지 데이터 -> 모델의 입력으로 변환

- Resize() : 256*256 크기로 데이터 조정
- RandomResizedCrop : 랜덤한 비율로 자른 후 데이터 크기를 (224*224)로 조정
- RandomHorizontalFlip : 이미지가 랜덤하게 좌우반전됨
- ToTensor : 이미지 데이터를 텐서로 변환

이미지 크기:

원본 이미지(x,y) -> (256,256) -> (224,224)

#5.3 전이 학습

#RandomResizedCrop 자세히 알아보기

Q)Resize() 후 RandomResizedCrop()을 하는 이유는?

A) 원본 이미지 크기가 제각각이기 때문에 바로 RandomResizedCrop()을 하면 이미지 크기가 너무 작거나 왜곡될 수 있기 때문에 Resize(256,256)으로 일정한 크기로 통일한 후, 적용

=> RandomResizedCrop을 통해서 데이터가 다양해지는 효과를 얻을 수 있음

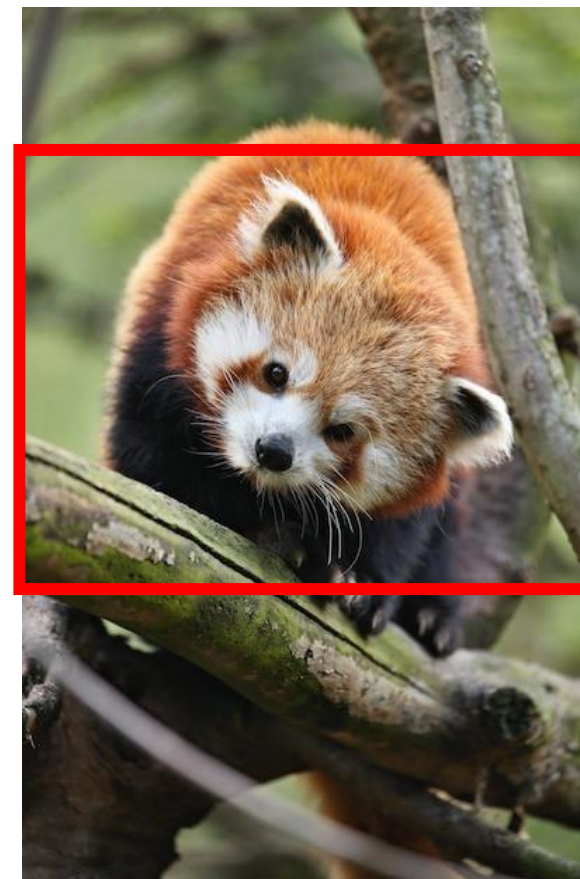
● `Transforms.RandomResizedCrop(size=(x,y), scale=(0.1,1), ratio=(0.5,2))`

size: 출력할 크기를 $x*y$ 로 조정

scale: 면적 비율을 0.1~1 (10~100%) 범위 내에서 무작위로 자름

ratio : 면적의 너비와 높이 비율을 0.5~2 범위 내에서 무작위로 조절

=> **ratio**를 통해 너비와 비율을 조절한 후, **scale**을 통해 자르고, 최종적으로 출력한 **size**로 조정



#5.3 전이 학습

#이미지 데이터 전처리 방법 정의

```
# 이미지 데이터 전처리 방법 정의
data_path = r'C:\Euron\Week4\catanddog\train'

transform = transforms.Compose(
    [
        transforms.Resize([256, 256]),
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor()
    ])

train_dataset = torchvision.datasets.ImageFolder(
    data_path,
    transform=transform
)

train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=32,
    num_workers=8,
    shuffle=True
)

print(len(train_dataset))
```

- datasets.ImageFolder : 데이터로더가 데이터를 불러올 대상과 방법 정의
 - 첫번째 파라미터: 불러올 데이터가 위치한 경로
 - transform: 이미지 데이터에 대한 전처리 정의

#5.3 전이 학습

#이미지 데이터 전처리 방법 정의

```
# 이미지 데이터 전처리 방법 정의
data_path = r'C:\Euron\Week4\catanddog\train'

transform = transforms.Compose(
    [
        transforms.Resize([256, 256]),
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor()
    ])

train_dataset = torchvision.datasets.ImageFolder(
    data_path,
    transform=transform
)

train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=32,
    num_workers=8,
    shuffle=True
)

print(len(train_dataset))
```

- **DataLoader** : 데이터를 불러오는 부분으로 앞에서 정의한 ImageFolder(train_dataset)을 데이터로더에 할당함
 - 첫번째 파라미터(train_dataset) : 데이터셋을 지정
 - batch_size : 한번에 불러올 데이터양을 결정하는 배치 크기를 설정
 - num_workers : 데이터를 불러올 때 하위 프로세스를 몇 개를 사용할지 설정 (너무 적게 설정하면 속도가 느리고, 너무 많이 설정하면 오류가 발생하거나 메모리 부족 현상 발생)
 - shuffle : 데이터를 무작위로 섞을지 지정
- **print(len(train_dataset))**: train_dataset에 포함된 데이터 개수
출력 = 385

#5.3 전이 학습

#학습에 사용될 이미지 출력

```
# 학습에 사용될 이미지 출력
data_iter = iter(train_loader)
samples, labels = next(data_iter)
classes = {0:'cat', 1:'dog'} # 개와 고양이에 대한 클래스로 구성
fig = plt.figure(figsize=(16,24))
for i in range(24): # 24개의 이미지 데이터 출력
    a = fig.add_subplot(4,6,i+1)
    a.set_title(classes[labels[i].item()]) # 레이블 정보와 함께 출력
    a.axis('off')
    a.imshow(np.transpose(samples[i].numpy(), (1,2,0)))

plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

- `iter(train_loader)` : `train_loader`를 이터레이터(반복자)로 변환
- `Next(data_iter)` : 이터레이터에서 다음 batch의 데이터를 가져오는 역할

Train_loader 에서 데이터를 samples와 labels 의 값을 순차적으로 꺼내 저장

#5.3 전이 학습

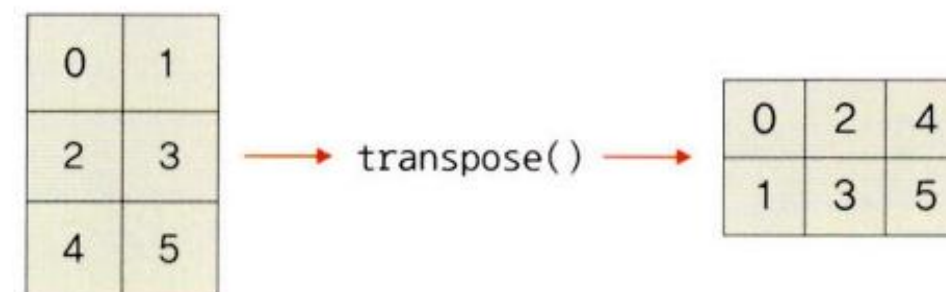
#학습에 사용될 이미지 출력

```
# 학습에 사용될 이미지 출력
data_iter = iter(train_loader)
samples, labels = next(data_iter)
classes = {0:'cat', 1:'dog'} # 개와 고양이에 대한 클래스로 구성
fig = plt.figure(figsize=(16,24))
for i in range(24): # 24개의 이미지 데이터 출력
    a = fig.add_subplot(4,6,i+1)
    a.set_title(classes[labels[i].item()]) # 레이블 정보와 함께 출력
    a.axis('off')
    a.imshow(np.transpose(samples[i].numpy(), (1,2,0)))

plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

- `np.transpose` : 행과 열을 바꿈으로써 행렬의 차원을 바꾸어 줌
행렬의 차원을 바꾸는 이유 = 행렬의 내적 연산
A와 B를 내적하려면 A 행렬의 행과 B 행렬의 열 수가 같아야 함

▼ 그림 5-33 transpose()



- `np.transpose(samples[i].numpy(), (1,2,0))` :
`samples[i].numpy()`는 PyTorch 텐서를 numpy 배열로 변경한 것
1은 원래 배열의 두번째 차원(높이)가 첫번째 위치로 이동한다는 것을 의미
2는 원래 배열의 세번째 차원(너비)가 두번째 위치로 이동한다는 것을 의미
0은 원래 배열의 첫번째 차원(채널)이 세번째 위치로 이동한다는 것을 의미
∴ (채널, 높이, 너비) -> (높이, 너비, 채널)

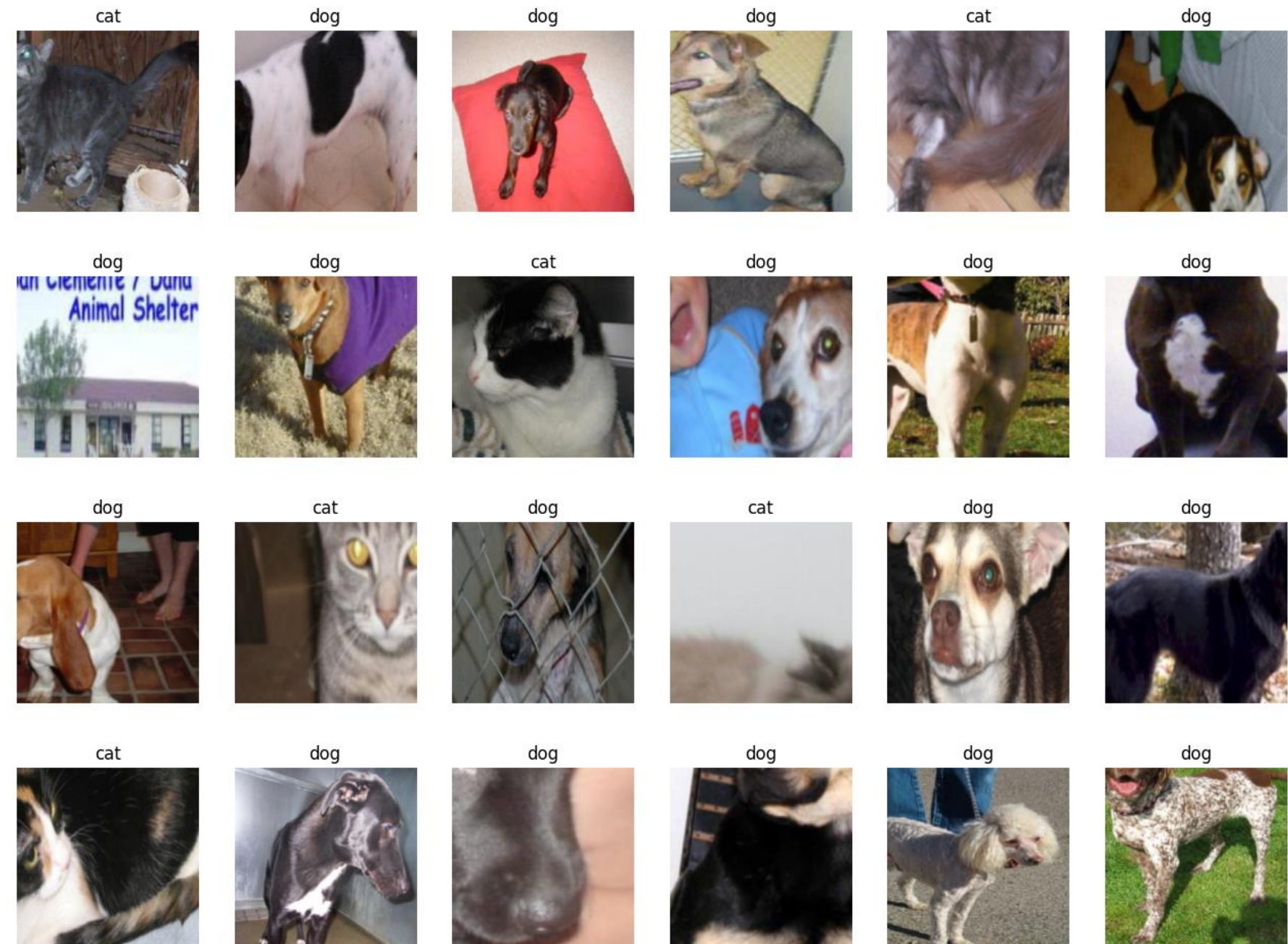
⇒ PyTorch는 이미지를 (채널, 높이, 너비) 형식으로 저장하지만, matplotlib은 (높이, 너비, 채널) 형식을 사용하기 때문에 다음과 같이 변경해주는 과정이 꼭 필요함

#5.3 전이 학습

#학습에 사용될 이미지 출력

```
# 학습에 사용될 이미지 출력
data_iter = iter(train_loader)
samples, labels = next(data_iter)
classes = {0:'cat', 1:'dog'} # 개와 고양이에 대한 클래스로 구성
fig = plt.figure(figsize=(16,24))
for i in range(24): # 24개의 이미지 데이터 출력
    a = fig.add_subplot(4,6,i+1)
    a.set_title(classes[labels[i].item()]) # 레이블 정보와 함께 출력
    a.axis('off')
    a.imshow(np.transpose(samples[i].numpy(), (1,2,0)))

plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```



#5.3 전이 학습

#사전 훈련된 모델 내려받기

```
# 사전 훈련된 모델 내려받기  
resnet18 = models.resnet18(pretrained=True)
```

- `models.resnet18()` : 파이토치에서 무작위의 가중치로 모델을 구성
- `pretrained = True` : 사전 학습된 가중치를 사용하겠다는 의미

●ResNet18

- 50개의 계층으로 구성된 합성곱 신경망
- ImageNet 데이터베이스로 훈련된 신경망으로, 학습에 사용되도록 사전 훈련된 모델 제공
- 입력 제약이 크고, 충분한 크기의 RAM이 없으면 학습 속도가 느릴 수 있다는 단점이 있음

5.3 전이학습 (2)



사전 훈련된 모델의 파라미터 학습 유무 지정

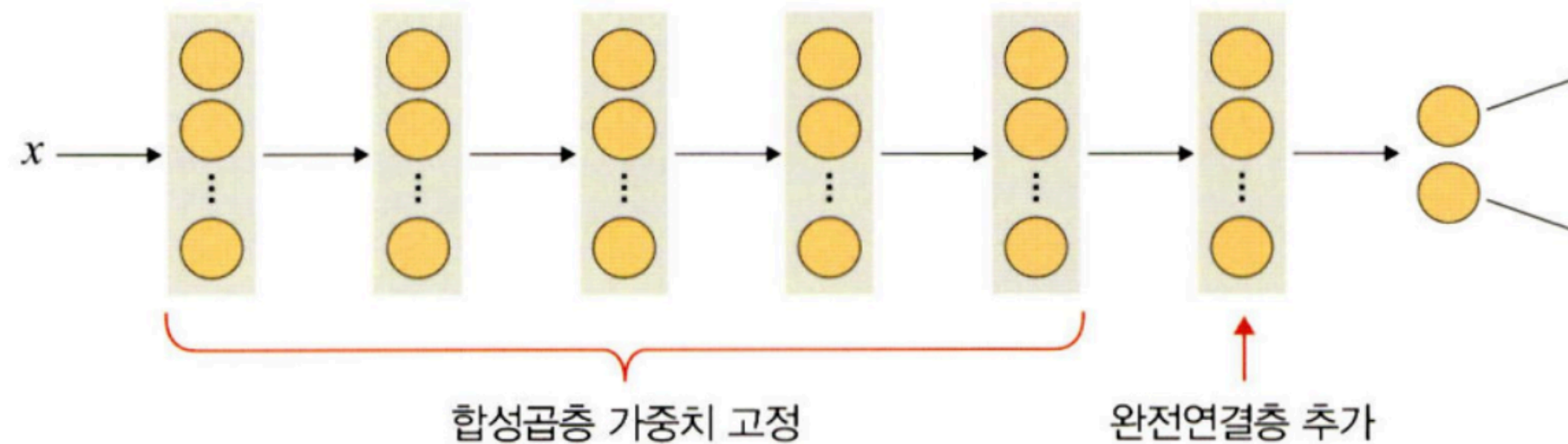
```
[ ] def set_parameter_requires_grad(model, feature_extracting=True):  
    if feature_extracting:  
        for param in model.parameters():  
            param.requires_grad = False  
  
set_parameter_requires_grad(resnet18)
```

- 내려받은 ResNet18의 합성곱층을 사용하되 파라미터에 대해서는 학습을 하지 않고 고정시킴
- **param.requires_grad = False**
 - 역전파 중 파라미터들에 대한 변화를 계산할 필요 없음
 - 모델의 일부(합성곱층 + 풀링층)을 고정하고 나머지를 학습하고자 할 때

ResNet18에 완전연결층 추가

```
[ ] resnet18.fc = nn.Linear(512,2) # 2 : 클래스가 (개, 고양이) 두 개라는 의미
```

▼ 그림 5-36 ResNet18에 완전연결층 추가



- 내려받은 ResNet18의 마지막 부분에 FC layer(완전연결층) 추가
- 추가된 완전연결층은 개와 고양이 클래스를 분류하는 용도로 사용

모델의 파라미터 값 확인

```
[ ] for name, param in resnet18.named_parameters(): # model.named_parameters() : 모델에 접근하여 파라미터 값 가져오기
    if param.requires_grad:
        print(name, param.data)
```

```
fc.weight tensor([[ 0.0290, -0.0368, -0.0243, ..., 0.0268, 0.0337, 0.0388],
                  [ 0.0173, -0.0082, 0.0215, ..., -0.0272, -0.0227, -0.0236]])
fc.bias tensor([-0.0427, -0.0383])
```

모델 객체 생성 및 손실 함수 정의

```
[ ] model = models.resnet18(pretrained=True) # 모델 객체 생성

for param in model.parameters(): # 모델의 합성곱층 가중치 고정
    param.requires_grad = False

model.fc = torch.nn.Linear(512,2)
for param in model.fc.parameters(): # 완전연결층은 학습
    param.requires_grad = True

optimizer = torch.optim.Adam(model.fc.parameters())
cost = torch.nn.CrossEntropyLoss() # 손실 함수 정의

print(model)
```

```
(avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
(fc): Linear(in_features=512, out_features=2, bias=True)
```

모델 학습을 위한 함수 생성

```
def train_model(model, dataloaders, criterion, optimizer, device, num_epochs=13, is_train=True):
    since = time.time() # 컴퓨터의 현재 시각을 구하는 함수
    acc_history = []
    loss_history = []
    best_acc = 0.0

    for epoch in range(num_epochs):
        print('Epoch {}/{}'.format(epoch, num_epochs-1))
        print('-'*10)

        running_loss = 0.0
        running_corrects = 0

        for inputs, labels in dataloaders : # 데이터로더에 전달된 데이터만큼 반복
            inputs = inputs.to(device)
            labels = labels.to(device)

            model.to(device)
            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            _, preds = torch.max(outputs, 1)
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * inputs.size(0) # 출력 결과와 레이블의 초차를 계산한 결과를 누적하여 저장
            running_corrects += torch.sum(preds == labels.data) # 출력 결과와 레이블이 동일한지 확인한 결과를 누적하여 저장

        epoch_loss = running_loss/ len(dataloaders.dataset) # 평균 오차 계산
        epoch_acc = running_corrects.double() / len(dataloaders.dataset) # 평균 정확도 계산

        print('Loss: {:.4f} Acc: {:.4f}'.format(epoch_loss,epoch_acc))

        if epoch_acc > best_acc:
            best_acc = epoch_acc

        acc_history.append(epoch_acc.item())
        loss_history.append(epoch_loss)
        torch.save(model.state_dict(), os.path.join('/content/catanddog/', '{0:0=2d}.pth'.format(epoch))) # 모델 재사용을 위해 저장해 둬
        print()

    time_elapsed = time.time() - since # 실행 시간(학습 시간)을 계산
    print('Training complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_elapsed % 60))
    print('Best Acc: {:.4f}'.format(best_acc))

    return acc_history, loss_history # 모델의 정확도와 오차를 반환
```

파라미터 학습 결과를 옵티마이저에 전달

```
[ ] params_to_update = []
for name, param in resnet18.named_parameters():
    if param.requires_grad == True:
        params_to_update.append(param) # 파라미터 학습 결과를 저장
        print("/t",name)

optimizer = optim.Adam(params_to_update) # 학습 결과를 옵티마이저에 전달
```

fc.weight
fc.bias

모델 학습

```
[ ] device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    criterion = nn.CrossEntropyLoss() # 손실 함수 지정
    train_acc_hist, train_loss_hist = train_model(resnet18, train_loader, criterion, optimizer, device)
```

Epoch 12/12

Loss: 0.1985 Acc: 0.9169

Training complete in 6m 27s

Best Acc: 0.937662

테스트 데이터 호출 및 전처리

```
[ ] test_path = '/content/catanddog/test'

    transform = transforms.Compose(
        [
            transforms.Resize(224),
            transforms.CenterCrop(224),
            transforms.ToTensor(),
        ])
    test_dataset = torchvision.datasets.ImageFolder(
        root=test_path,
        transform=transform
    )
    test_loader = torch.utils.data.DataLoader(
        test_dataset,
        batch_size=32,
        num_workers=1,
        shuffle=True
    )

    print(len(test_dataset))
```

테스트 데이터 평가 함수 생성

```
[ ] def eval_model(model, dataloaders, device):
    since = time.time()
    acc_history = []
    best_acc = 0.0

    saved_models = glob.glob('catanddog/' + '*.pth')
    saved_models.sort() # 불러온 .pth 파일을 정렬
    print('saved_model', saved_models)

    for model_path in saved_models:
        print('Loading model', model_path)

        model.load_state_dict(torch.load(model_path))
        model.eval()
        model.to(device)
        running_corrects = 0

        for inputs, labels in dataloaders: # 테스트 반복
            inputs = inputs.to(device)
            labels = labels.to(device)

            with torch.no_grad(): # autograd를 사용하지 않겠다는 의미
                outputs = model(inputs) # 데이터를 모델에 적용한 결과를 outputs에 저장

            _, preds = torch.max(outputs.data, 1)
            preds[preds >= 0.5] = 1 # torch.max로 출력된 값이 0.5보다 크면 올바르게 예측
            preds[preds < 0.5] = 0 # torch.max로 출력된 값이 0.5보다 작으면 틀리게 예측
            running_corrects += preds.eq(labels).int().sum()

        epoch_acc = running_corrects.double() / len(dataloaders.dataset) # 테스트 데이터의 정확도 계산
        print('Acc: {:.4f}'.format(epoch_acc))

        if epoch_acc > best_acc:
            best_acc = epoch_acc

        acc_history.append(epoch_acc.item())
        print()

    time_elapsed = time.time() - since
    print('Validation complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_elapsed % 60))
    print('Best Acc: {:.4f}'.format(best_acc))

    return acc_history # 계산된 정확도 반환
```

- **glob** : 현재 디렉터리에서 원하는 파일들만 추출하여 가져올 때 사용
 - 즉 '/content/catanddog/test' 경로에서 pth 확장자를 갖는 파일을 가져오라는 의미
 - .pth 라는 확장자를 갖는 파일 : 훈련 데이터로 모델을 훈련시킬 때 생성된 파일
- **torch.max** : 주어진 텐서 배열의 최댓값이 들어 있는 index 를 반환하는 함수
 - y_pred = [[0.2,0.7,0.8,0.4]]라는 배열에 torch.max(y_pred.data, 1) 을 적용한다고 가정
 - [0.2,0.7,0.8,0.4]의 최댓값은 0.8 이 되며 그 인덱스는 2 -> 최종적으로 반환되는 값은 2
- **preds.eq(1abels)** : preds 배열과 labels가 일치하는지 검사하는 용도
 - 뒤에 사용된 sum() : 모델의 예측 결과와 정답(레이블)이 일치하는 것들의 개수 합을 숫자로 출력

테스트 데이터를 평가 함수에 적용

```
[ ] val_acc_hist = eval_model(resnet18, test_loader, device)
```

```
Loading model ../chap05/data/catanddog\12.pth
Acc: 0.9286
```

```
Validation complete in 1m 52s
Best Acc: 0.948980
```

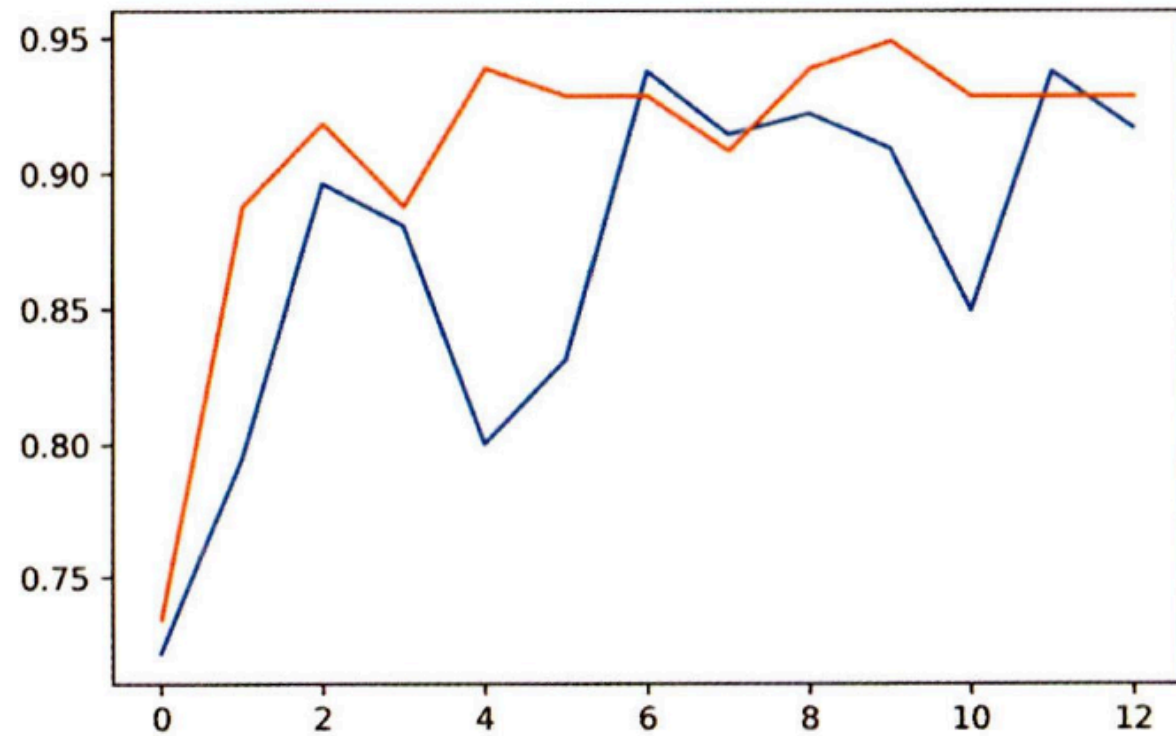
94% 정도의 높은 정확도

그래프 확인

훈련과 테스트 데이터의 정확도를 그래프로 확인

```
[ ] plt.plot(train_acc_hist)
plt.plot(val_acc_hist)
plt.show()
```

▼ 그림 5-37 훈련과 테스트 데이터에 대한 정확도

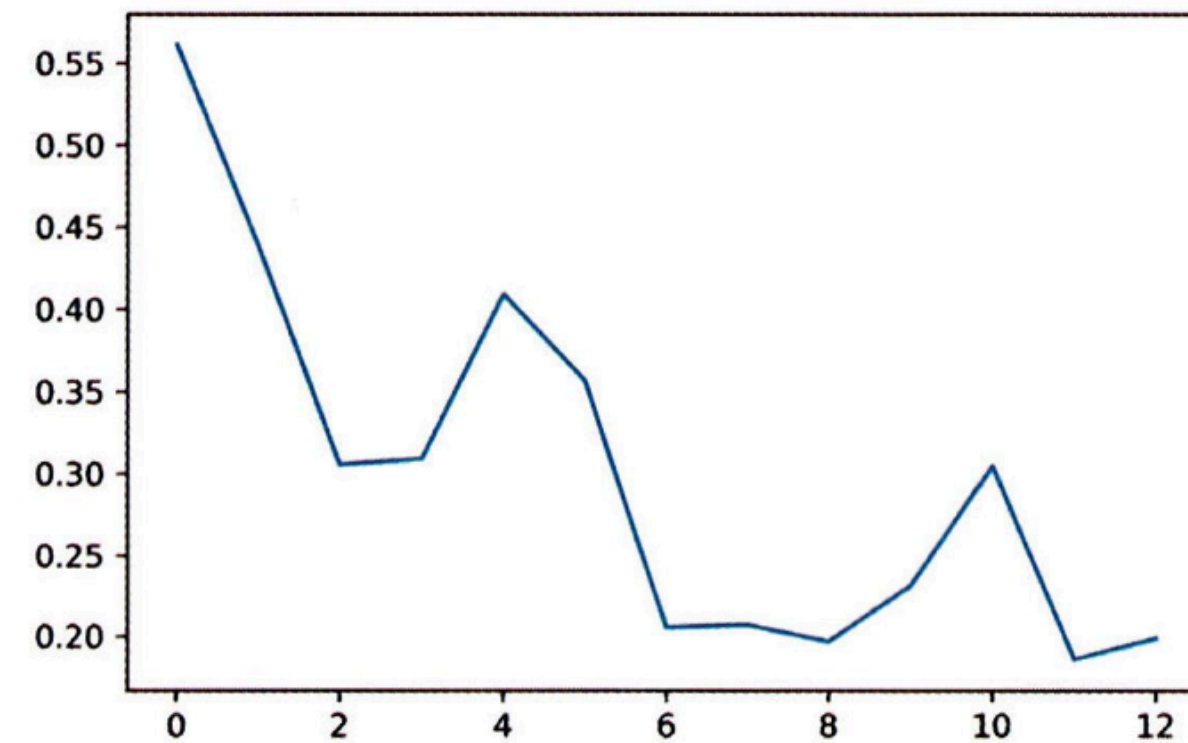


에포크가 진행될수록 정확도가 높아지면서 100% 가까워짐

훈련 데이터의 오차에 대한 그래프 확인

```
[ ] plt.plot(train_loss_hist)
plt.show()
```

▼ 그림 5-38 훈련 데이터에 대한 오차



에포크가 진행될수록 오차가 낮아지고 있음

예측 이미지 출력을 위한 전처리 함수

```
[ ] def im_convert(tensor):
    image=tensor.clone().detach().numpy()
    image=image.transpose(1,2,0)
    image=image*(np.array((0.5,0.5,0.5))+np.array((0.5,0.5,0.5)))
    image=image.clip(0,1)
    return image
```

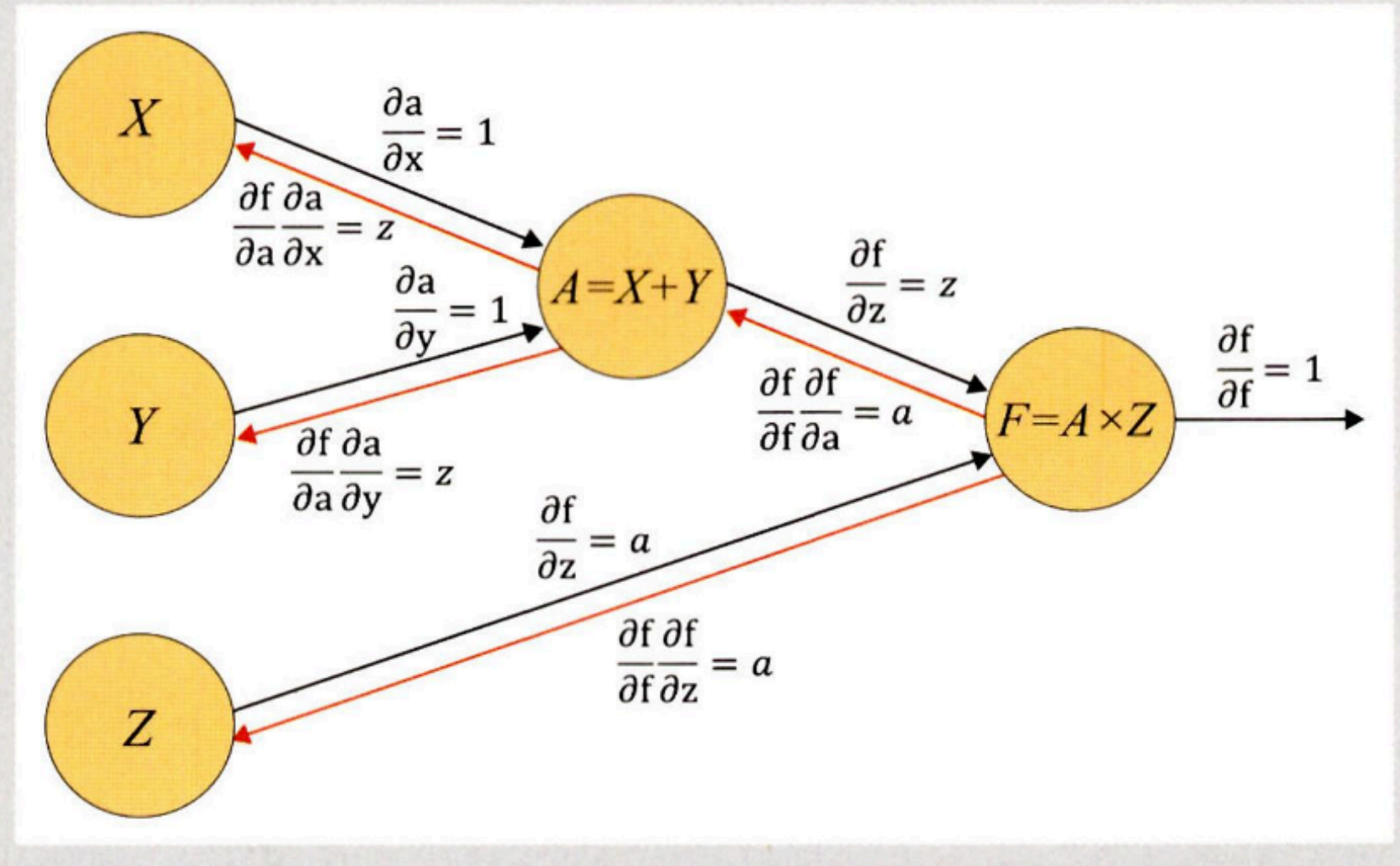
▼ 표 5-2 tensor.clone(), tensor.detach(), tensor.clone().detach()의 비교

구분	메모리	계산 그래프 상주 유무
tensor.clone()	새롭게 할당	계산 그래프에 계속 상주
tensor.detach()	공유해서 사용	계산 그래프에 상주하지 않음
tensor.clone().detach()	새롭게 할당	계산 그래프에 상주하지 않음

- `tensor.clone()` : 기존 텐서의 내용을 복사한 텐서를 생성하겠다는 의미
- `detach()` : 기존 텐서에서 기울기가 전파되지 않는 텐서
- `tensor.clone(). detach()` : 기존 텐서를 복사한 새로운 텐서를 생성하지만 기울기에 영향을 주지는 않겠다는 의미
- `clip()` : 입력 값이 주어진 범위를 벗어날 때 입력 값을 특정 범위로 제한시키기 위해 사용
 - `image.clip(0, 1)` : image 데이터를 0과 1 사이의 값으로 제한하겠다는 의미

계산 그래프

▼ 그림 5-39 계산 그래프



계산 그래프를 사용하는 이유

- 국소적 계산이 가능
- 전파를 통한 미분 계산이 편리

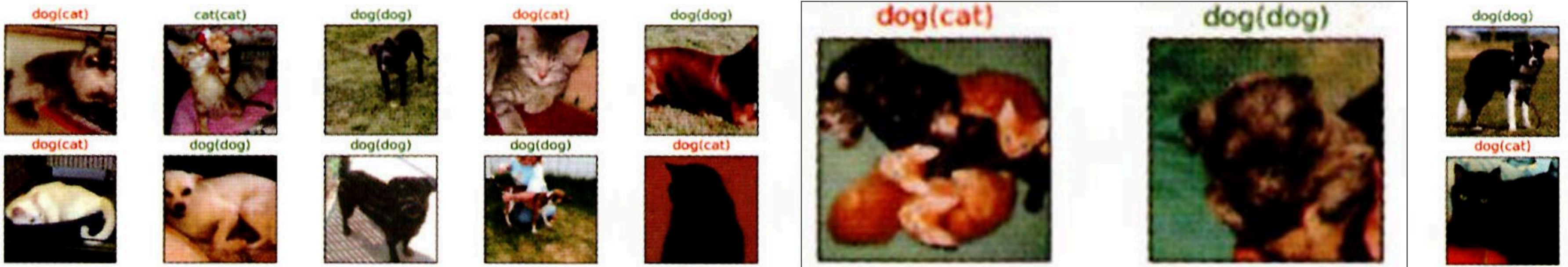
- 계산 과정을 그래프로 나타낸 것
- 여러 개의 노드 (node)와 그 노드들을 연결하는 선인 에지 (edge)로 구성

개와 고양이 예측 결과 출력

```
▶ classes = {0:'cat', 1:'dog'}

dataiter=iter(test_loader)
images, labels = next(dataiter)
output=model(images)
_,preds=torch.max(output,1)

fig=plt.figure(figsize=(25,4))
for idx in np.arange(20):
    ax=fig.add_subplot(2,10,idx+1,xticks=[],yticks=[])
    plt.imshow(im_convert(images[idx]))
    a.set_title(classes[labels[i].item()])
    ax.set_title("{}({})".format(str(classes[preds[idx].item()]),str(classes[labels[idx].item()])),color=("green" if preds[idx]==labels[idx] else "red"))
plt.show()
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

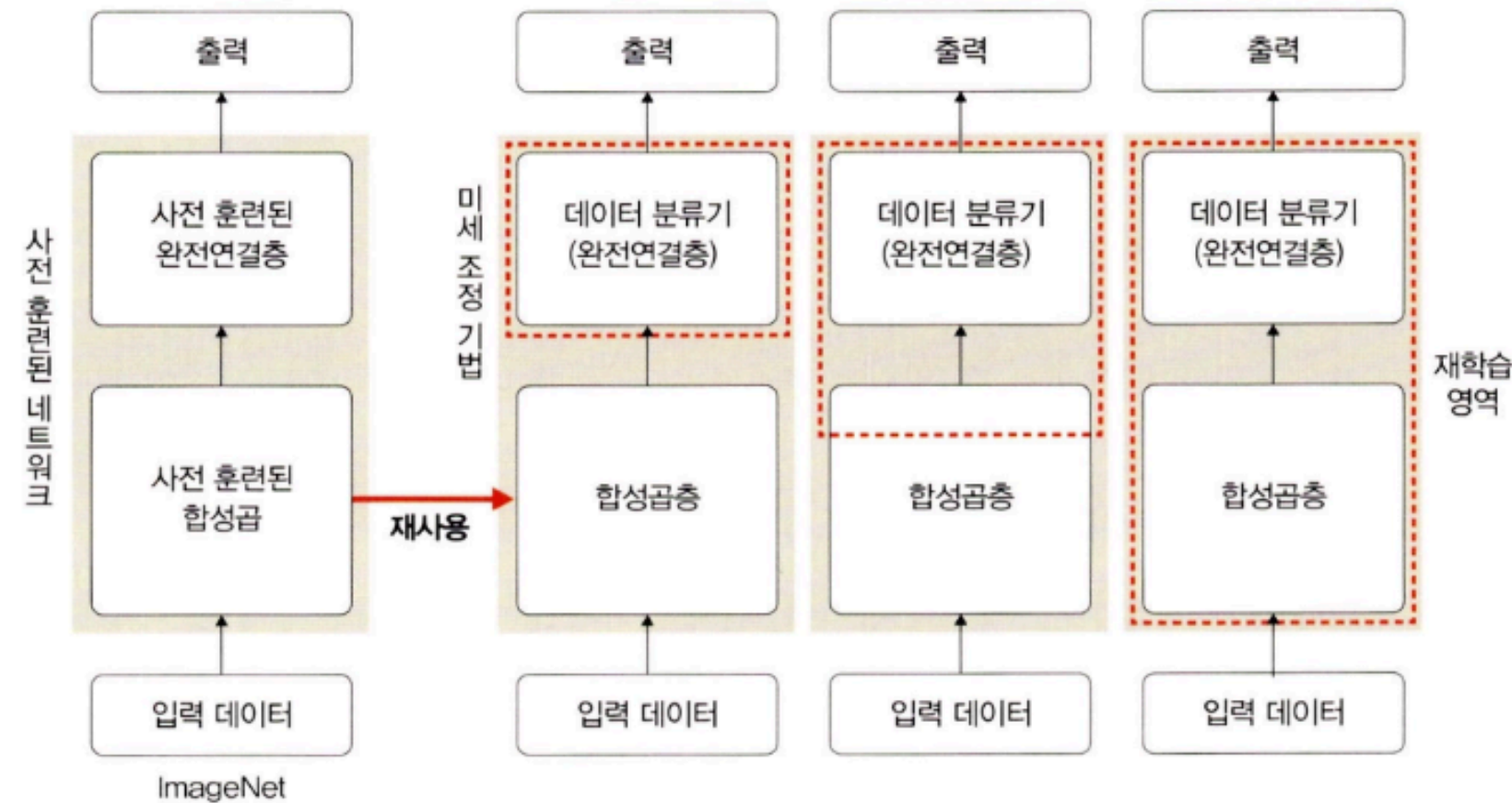


- 초록색은 개와 고양이를 정확하게 예측한 것이고, 빨간색은 예측이 잘못되었음을 의미
- 결과를 살펴보니 예측이 정확하지 않은 것을 확인할 수 있음
-> 훈련 데이터를 더 늘리고, 에포크 횟수도 늘려 보면 더 좋은 결과를 얻을 수 있음

EWCHA
EUROPEAN

- 다음 그림과 같이 Figure 안에서 서브플롯(subplot)(다음 그림에서 Axes에 해당)의 위치를 조정할 때 사용
 - left, bottom, right, top으로 이미지 위치를 조정
 - hspace와 wspace를 사용해서 서브플롯 간의 간격(너비와 높이의 비율)을 조정할 수 있음

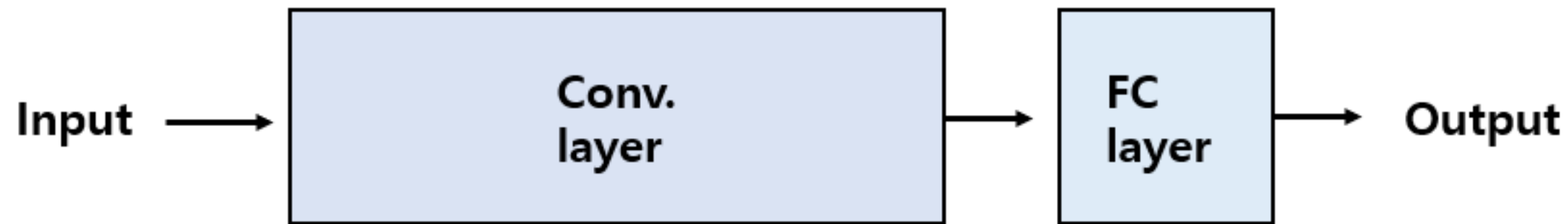
5.3.2 미세 조정 기법

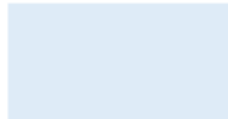


- 사전 훈련된 모델과 합성곱층, 데이터 분류기의 가중치를 업데이트하여 훈련시키는 방식
 - 목적에 맞게 재학습시키거나 학습된 가중치의 일부를 재학습
 - 미세 조정하여 분석하려는 데이터셋에 잘 맞도록 모델의 파라미터를 조정하는 기법
- 미세 조정 과정에서 많은 연산량이 필요하기 때문에 GPU 사용 권장
- 미세 조정은 파라미터 업데이트 과정에서 파라미터에 큰 변화를 주게 되면 과적합 문제가 발생할 수 있기 때문에 정교하고 미세한 파라미터 업데이트가 필요함

데이터셋의 크기와 사전 훈련된 모델에 따른 미세 조정 기법 전략

1. 데이터셋이 크고 사전 훈련된 모델과 유사성이 작을 경우



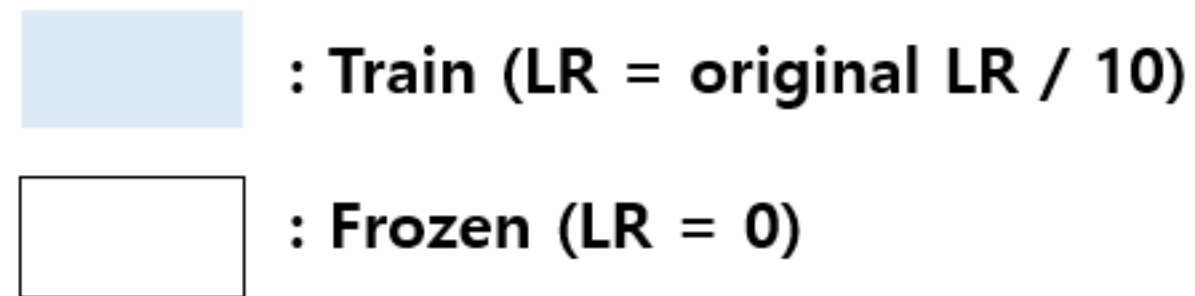
 : Train (LR = original LR / 10)

 : Frozen (LR = 0)

- 새로 학습시킬 데이터셋이 많다면, overfitting 위험이 낮기 때문에
- pre-trained model을 포함 전체 네트워크 fine-tuning 추천

데이터셋의 크기와 사전 훈련된 모델에 따른 미세 조정 기법 전략

2. 데이터셋이 크고 사전 훈련된 모델과 유사성이 클 경우

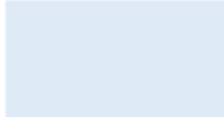


- pre-trained model의 일부(합성곱층의 완전연결층과 가까운 부분)과 완전연결층(데이터분류기, FC layer)를 학습시킴
- 데이터셋이 유사하기 때문에 전체를 학습시키는 것 보다 강한 특징이 나타나는 합성곱층 뒷부분과 완전연결층만 새로 학습해도 좋은 성능을 낼 수 있음

데이터셋의 크기와 사전 훈련된 모델에 따른 미세 조정 기법 전략

3. 데이터셋이 작고 사전 훈련된 모델과 유사성이 작을 경우



 : Train (LR = original LR / 10)

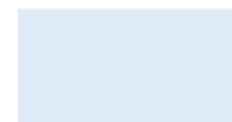
 : Frozen (LR = 0)

- 합성곱층의 일부분과 완전연결층(데이터 분류기, FC layer)를 학습시킴
- 데이터가 적기 때문에 많은 계층에 미세 조정 기법을 적용하면 overfitting이 발생할 수 있음

데이터셋의 크기와 사전 훈련된 모델에 따른 미세 조정 기법 전략

4. 데이터셋이 작고 사전 훈련된 모델과 유사성이 클 경우



 : Train (LR = original LR / 10)
 : Frozen (LR = 0)

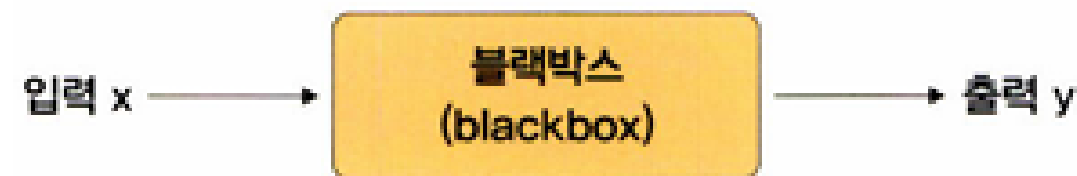
- 완전연결층(데이터 분류기, FC layer)만 학습시킴

5.4 설명 가능한 CNN



설명 가능한 CNN

설명 가능한 CNN: 딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 제시하는 기술



CNN은 블랙박스와 같음 -> 신뢰하기가 어려움 -> 처리 과정을 시각화 함으로써 해결!
CNN 시각화 방법: 필터에 대한 시각화 & 특성 맵에 대한 시각화

특성 맵: 입력 이미지 또는 다른 특성 맵처럼 필터를 입력에 적용한 결과
→ 시각화? 특성 맵에서 입력 특성을 감지하는 방법을 이해

특성 맵 시각화 (예제)

필요한 라이브러리 호출

```
import matplotlib.pyplot as plt
from PIL import Image
import cv2
import torch
import torch.nn.functional as F
import torch.nn as nn
from torchvision.transforms import ToTensor
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models

device=torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

PIL: 이미지 분석 및 처리를 쉽게 할 수 있도록 도와주는 라이브러리
➔ 다양한 이미지 파일 형식 지원, 강력한 이미지 처리와 그래픽 기능 제공

```
> pip install pillow
```

특성 맵 시각화 (예제)

13개의 합성곱층과 두 개의 완전 연결층으로 구성된 네트워크 생성

```
class XAI(torch.nn.Module):
    def __init__(self, num_classes=2):
        super(XAI, self).__init__()
        self.features=nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.Dropout(0.3),
            nn.Conv2d(64, 64, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(128, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(128, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),
```

```
            nn.Conv2d(256, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),
        )
        self.classifier=nn.Sequential(
            nn.Linear(512, 512, bias=False),
            nn.Dropout(0.5),
            nn.BatchNorm1d(512),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(512, num_classes)
        )
```

```
def forward(self, x):
    x=self.features(x)
    x=x.view(-1, 512)
    x=self.classifier(x)
    return F.log_softmax(x)
```

특성 맵 시각화 (예제)

`log_softmax()`: 신경망 말단의 결과 값들을 확률 개념으로 해석하기 위해 소프트맥스 함수의 결과에 `log` 값을 취한 연산

➔ 왜 `log` 값? 소프트맥스는 기울기 소멸 문제에 취약하기 때문

```
def forward(self, x):  
    x=self.features(x)  
    x=x.view(-1, 512)  
    x=self.classifier(x)  
    return F.log_softmax(x)
```

- 소프트맥스 수식

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1} e^{x_j}}$$

로그 취하고 식 정리



- 로그 소프트맥스 수식

$$\begin{aligned}\text{logsoftmax} &= \log\left(\frac{e^{x_i}}{\sum_{j=1} e^{x_j}}\right) \\ &= x_i - \log\left(\sum_{j=1} e^{x_j}\right)\end{aligned}$$

특성 맵 시각화 (예제)

앞서 생성한 모델을 model=XAI()로 객체화, 장치에 할당

```
model=XAI() #model이라는 이름의 객체 생성
model.to(device) #model을 장치(CPU/GPU)에 할당
model.eval() #테스트 데이터에 대한 모델 평가 용도
```

```
XAI(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), bias=False)
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.3, inplace=False)
    (4): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): ReLU(inplace=True)
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): ReLU(inplace=True)
    (11): Dropout(p=0.4, inplace=False)
    (12): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (13): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (14): ReLU(inplace=True)
    (15): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (16): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (17): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (18): ReLU(inplace=True)
    (19): Dropout(p=0.4, inplace=False)
    (20): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (21): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (22): ReLU(inplace=True)
    (23): Dropout(p=0.4, inplace=False)
    (24): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (25): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (26): ReLU(inplace=True)
    (27): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (28): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (29): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (30): ReLU(inplace=True)
    (31): Dropout(p=0.4, inplace=False)
    (32): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (33): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (34): ReLU(inplace=True)
    (35): Dropout(p=0.4, inplace=False)
    (36): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (37): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (38): ReLU(inplace=True)
    (39): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (40): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (41): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (42): ReLU(inplace=True)
    (43): Dropout(p=0.4, inplace=False)
    (44): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (45): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (46): ReLU(inplace=True)
    (47): Dropout(p=0.4, inplace=False)
    (48): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (49): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (50): ReLU(inplace=True)
    (51): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Sequential(
    (0): Linear(in_features=512, out_features=512, bias=False)
    (1): Dropout(p=0.5, inplace=False)
    (2): BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (3): ReLU(inplace=True)
    (4): Dropout(p=0.5, inplace=False)
    (5): Linear(in_features=512, out_features=2, bias=True)
  )
)
```

특성 맵 시각화 (예제)

특성 맵을 확인하기 위한 클래스 정의

- (0): Conv2d(3, 64, kernel_size=(3,3), stride=(1,1), bias=False에 대한 특성 맵

```
class LayerActivations:
    features=[]
    def __init__(self, model, layer_num):
        self.hook=model[layer_num].register_forward_hook(self.hook_fn)
    def hook_fn(self, module, input, output):
        self.features=output.detach().numpy()
    def remove(self):
        self.hook.remove()
```

#hook을 통해 각 계층의 활성화 함수 및 기울기 값 확인 가능

- **register_forward_hook**: 순전파 중에 각 네트워크 모듈의 입력/출력 가져오기

Ex)

```
import torch
x=torch.Tensor([0,1,2,3]).requires_grad_()
y=torch.Tensor([4,5,6,7]).requires_grad_()
w=torch.Tensor([1,2,3,4]).requires_grad_()
z=x+y;
o=w.matmul(z)
o.backward()
print(x.grad, y.grad, z.grad, w.grad, o.grad)
```

o와 z: 특정 값으로 정의 X, 중간 변수
- z.register_hook(hook_fn)을
이용하면 기울기 값 알기 가능!

```
tensor([1., 2., 3., 4.]) tensor([1., 2., 3., 4.]) None tensor([ 4.,  6.,  8., 10.]) None
```


특성 맵 시각화 (예제)

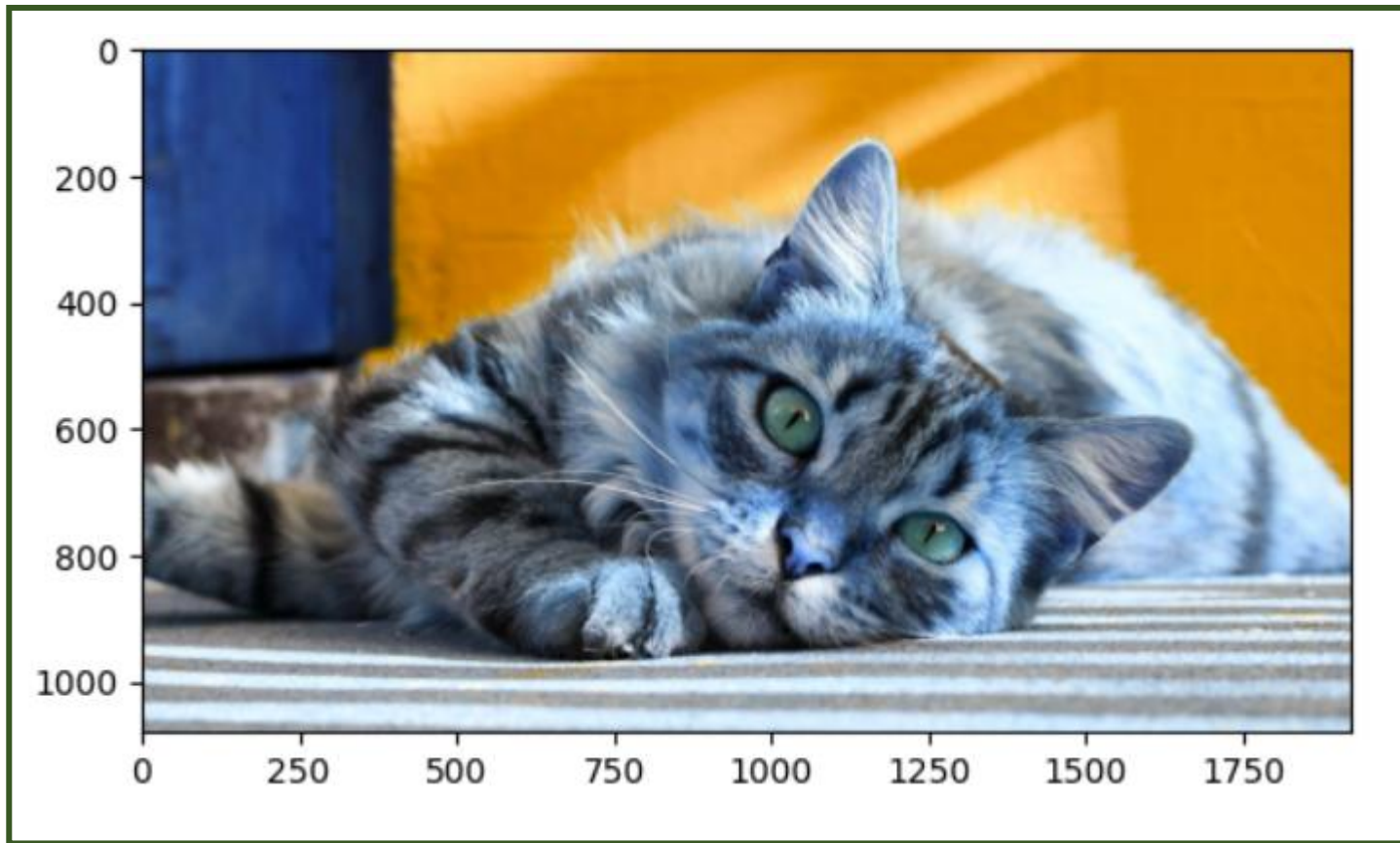
이미지 호출

```
img=cv2.imread('cat.jpg')
plt.imshow(img)
img=cv2.resize(img, (100,100), interpolation=cv2.INTER_LINEAR)
img=ToTensor()(img).unsqueeze(0)
print(img.shape)
```

#unsqueeze(): 1차원 데이터 생성

보간법: 이미지상에 존재하는 픽셀 데이터들에
근사 함수를 적용해서 새로운 픽셀 값을 구하는 것

#cv2.resize: 이미지 크기를 변경할 때 사용



Ex)

```
import torch
```

```
x1=torch.rand(3,10,64)
x2=x1.unsqueeze(dim=0)
print(x2.shape)
```

```
x3=x1.unsqueeze(dim=1)
print(x3.shape)
```

```
torch.Size([1, 3, 10, 64])
torch.Size([3, 1, 10, 64])
```

특성 맵 시각화 (예제)

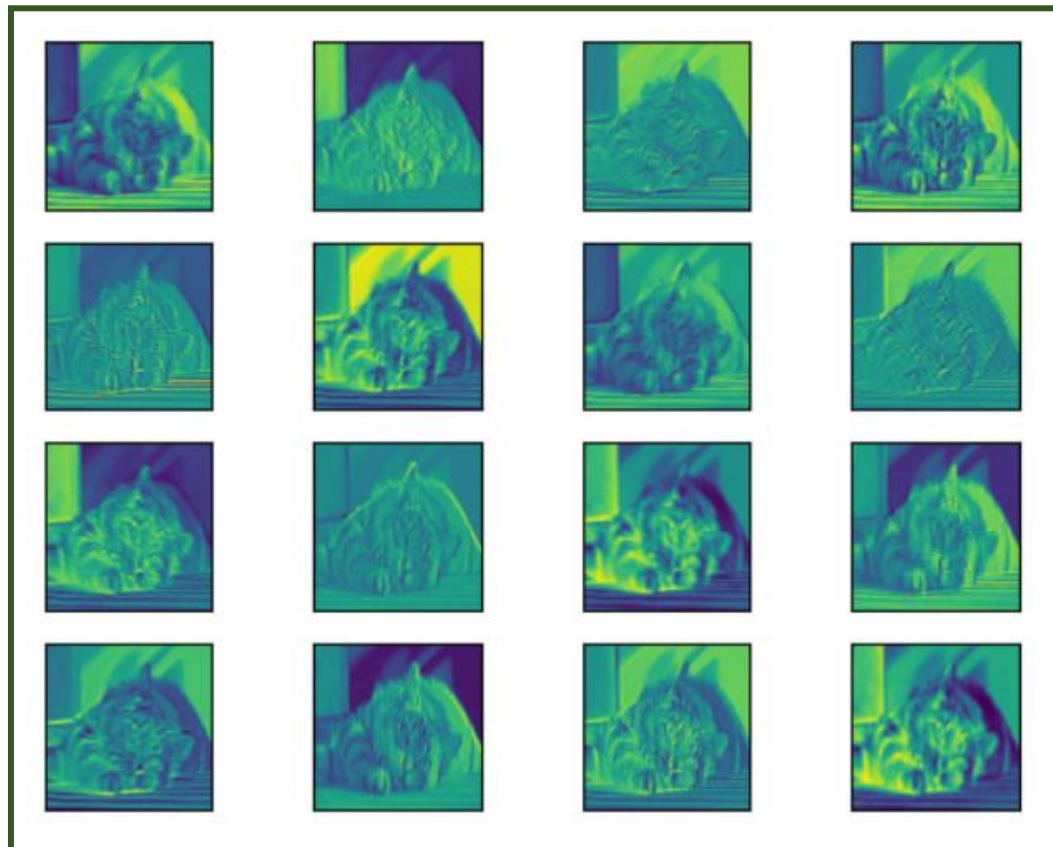
특성 맵 확인

```
result=LayerActivations(model.features,0)  
  
model(img)  
activations=result.features
```

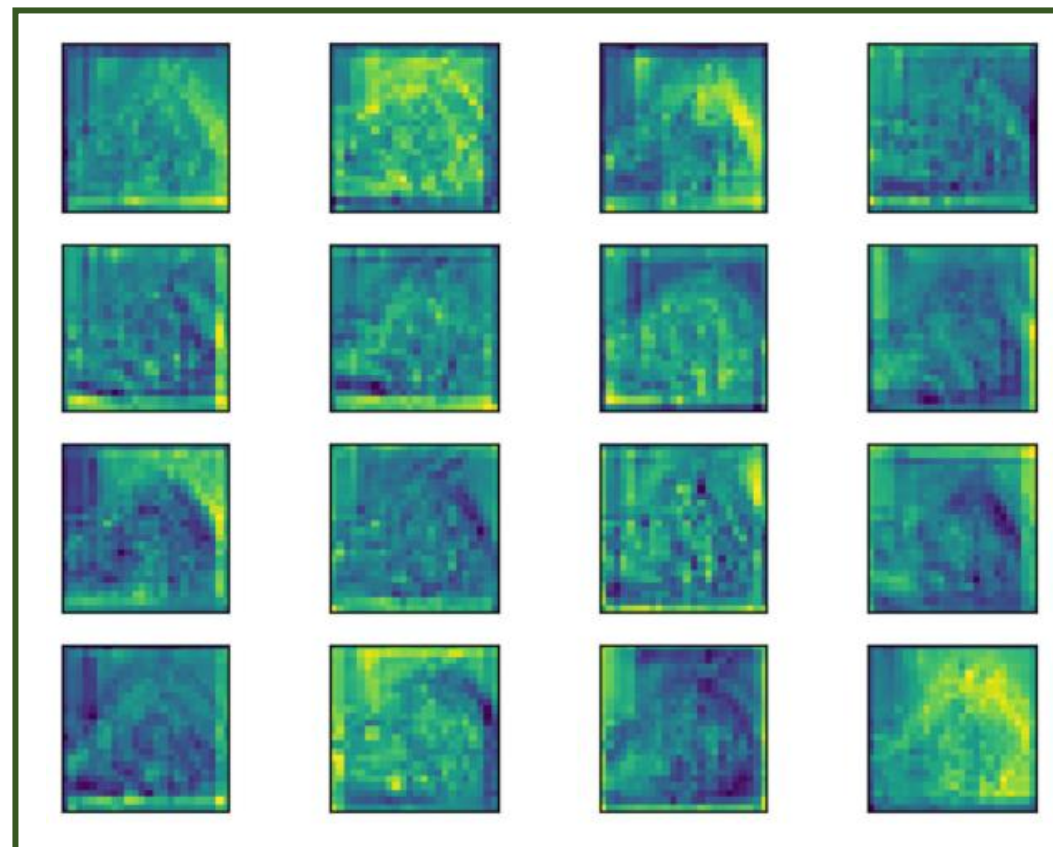
#각각 0, 20, 40 대입

```
fig, axes=plt.subplots(4,4)  
fig=plt.figure(figsize=(12,8))  
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)  
  
for row in range(4):  
    for column in range(4):  
        axis=axes[row][column]  
        axis.get_xaxis().set_ticks([])  
        axis.get_yaxis().set_ticks([])  
        axis.imshow(activations[0][row*10+column])  
plt.show()
```

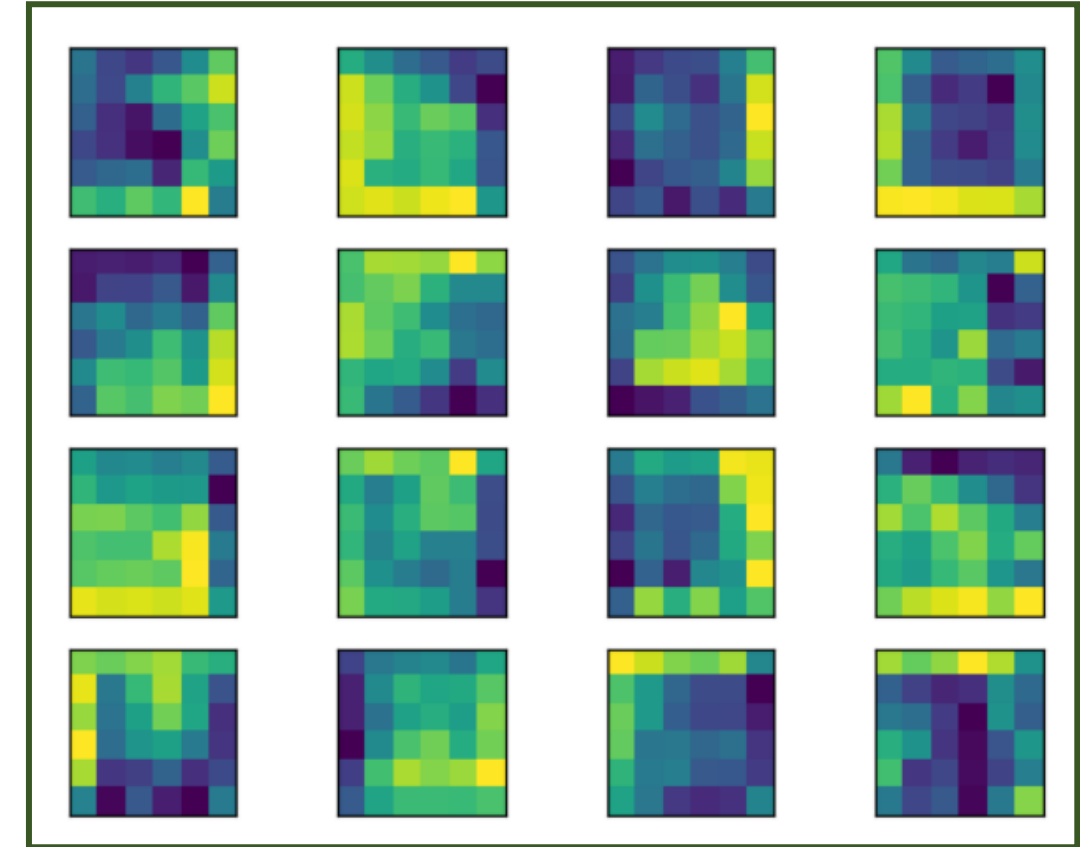
<0번째 계층>



<20번째 계층>



<40번째 계층>



5.5 그래프 합성곱 네트워크

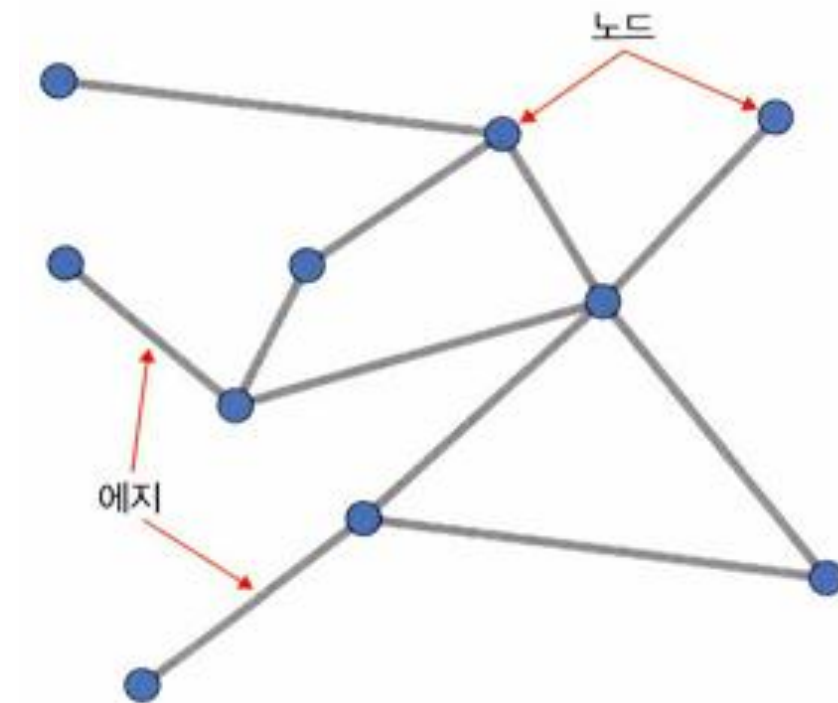


그래프 합성곱 네트워크

그래프 합성곱 네트워크: 그래프 데이터를 위한 신경망

그래프: 방향성이 있거나 없는 **에지**로 연결된 **노드**의 집합
(결합 방법) (원소)

그래프 신경망(GNN): 그래프 구조에서 사용하는 신경망



그래프 데이터에 대한 표현

그래프 데이터에 대한 표현

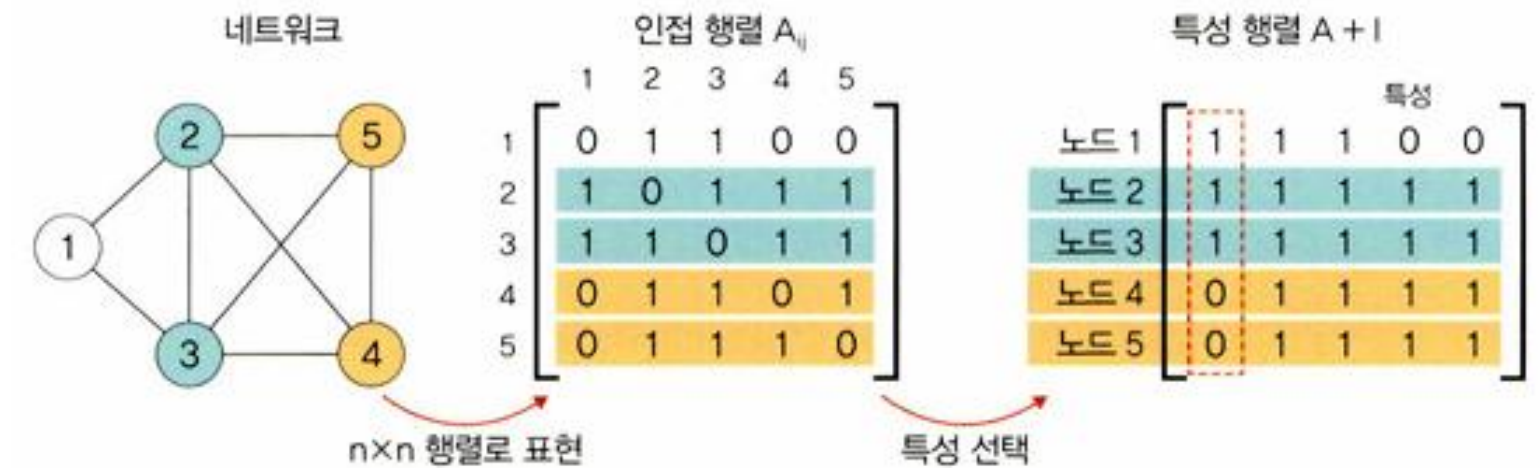
1단계. 인접 행렬

- 네트워크가 있을 때 노드 n 개를 $n \times n$ 행렬로 표현
- 인접 행렬 내의 값: ' A_{ij} 는 i 와 j 의 만족성 여부'

2단계. 특성 행렬

- 인접 행렬만으로는 특성 파악이 어려워 단위 행렬 적용
- 각 입력 데이터에서 이용할 특성 선택
- 각 행은 선택된 특성에 대해 각 노드가 갖는 값

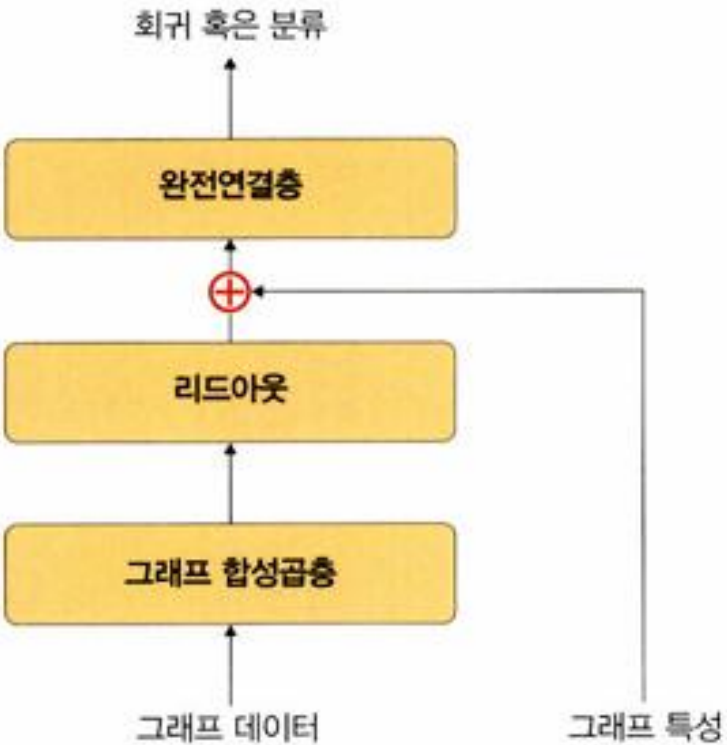
➔ 특성 행렬 과정을 거쳐 그래프 특성이 추출!!



그래프 합성곱 네트워크

그래프 합성곱 네트워크(GCN): 이미지에 대한 합성곱을 그래프 데이터로 확장한 알고리즘

그래프 합성곱 네트워크 구조



리드아웃: 특성 행렬을 하나의 벡터로 변환하는 함수

- 전체 노드의 특성 벡터에 대해 평균을 구하고 그래프 전체를 표현하는 하나의 벡터를 생성

그래프 합성곱층: 이를 이용한 그래프 형태의 데이터는 행렬 형태의 데이터로 변환, 딥러닝 알고리즘 적용 가능

활용 사례: SNS 관계 네트워크, 학술 연구의 인용 네트워크, 3D Mesh 등

THANK YOU

